Week03

本周作业：跑通赛题 → 编译优化 → perf 看热点

1. 跑通赛题

——源码包在组织仓库 HZCU-HPC-Team/2022_IPCC（2022_IPCC/pivot.tar）；
——缺什么先查清：已经有了吗（which / dpkg -l）？有没有同类工具？有没有更新的版本？查清楚再装；

< 1 它俩是通用的查询命令

which <命令名>：查**任何**可执行程序装没装、在哪。which gcc、which python3、which tmux 都行，赛题里用 gcc 只是例子。

dpkg -l：查**任何** deb 安装包装没装、什么版本。不只是“安装包工具”本身，而是“查所有已安装的包”——库（libssl-dev）、运行时（python3）都算。

```
Last login: Mon Nov  2 18:27:40 on ttys000
linguijun@linguijundeMacBook-Pro ~ % which gcc
/usr/bin/gcc
linguijun@linguijundeMacBook-Pro ~ % gcc --version
Apple clang version 21.0.0 (clang-2100.1.1.101)
Target: arm64-apple-darwin25.5.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
linguijun@linguijundeMacBook-Pro ~ % dpkg -l | grep build-essential
zsh: command not found: dpkg
linguijun@linguijundeMacBook-Pro ~ %
```

< 2

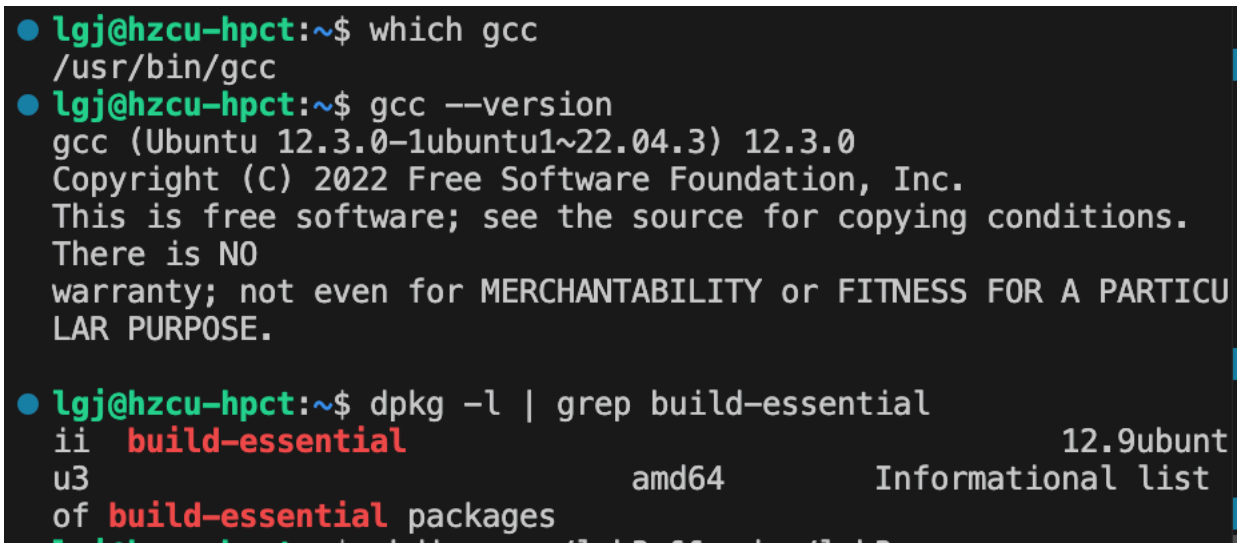
macOS 没有 dpkg 这个东西。dpkg 是 Debian/Ubuntu 的包管理器，Mac 是 Darwin 系统（查包：brew list | grep gcc——但没必要，clang 都 21 了，很新。）

< 3

命令输入应该在服务器上，在本地只能算练习，在终端通过输入服务器地址再输命令

**我选择直接vs code免密登录，新建终端（不记得服务器地址）

&环境体验


lgj@hzcu-hpct:~$ which gcc
/usr/bin/gcc
lgj@hzcu-hpct:~$ gcc --version
gcc (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
Copyright (C) 2022 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.
There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICU
LAR PURPOSE.
lgj@hzcu-hpct:~$ dpkg -l | grep build-essential
ii build-essential 12.9ubunt
u3 amd64 Informational list
of build-essential packages

&建工作目录，拿源码包

```
lgj@hzcu-hpct:~$ mkdir -p ~/lab3 && cd ~/lab3
lgj@hzcu-hpct:~/lab3$ cp /public1/soft/IPCC/2022/first/pivot.tar
cp: missing destination file operand after '/public1/soft/IPCC/2022/first/pivot.tar'
Try 'cp --help' for more information.
lgj@hzcu-hpct:~/lab3$ cp --help
Usage: cp [OPTION]... [-T] SOURCE DEST
  or:  cp [OPTION]... SOURCE... DIRECTORY
  or:  cp [OPTION]... -t DIRECTORY SOURCE...
Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.

Mandatory arguments to long options are mandatory for short options too.
  -a, --archive                same as -dR --preserve=all
      --attributes-only        don't copy the file data, just the attributes
      --backup[=CONTROL]       make a backup of each existing destination file
  -b                           like --backup but does not accept an argument
      --copy-contents          copy contents of special files when recursive
  -d                           same as --no-dereference --preserve=links
  -f, --force                  if an existing destination file cannot be
                                 opened, remove it and try again (this option
                                 is ignored when the -n option is also used)
  -i, --interactive            prompt before overwrite (overrides a previous -n
                                  option)
  -H                           follow command-line symbolic links in SOURCE
  -l, --link                   hard link files instead of copying
  -L, --dereference            always follow symbolic links in SOURCE
  -n, --no-clobber             do not overwrite an existing file (overrides
                                 a previous -i option)
  -P, --no-dereference         never follow symbolic links in SOURCE
  -p                           same as --preserve=mode,ownership,timestamps
      --preserve[=ATTR_LIST]   preserve the specified attributes (default:
                                 mode,ownership,timestamps), if possible
                                 additional attributes: context, links, xattr,
                                 all
      --no-preserve=ATTR_LIST  don't preserve the specified attributes
      --parents                use full source file name under
```

```
DIRECTORY
  -R, -r, --recursive          copy directories recursively
      --reflink[=WHEN]         control clone/CoW copies. See be
low
      --remove-destination     remove each existing destination
 file before
                                 attempting to open it (contras
t with --force)
      --sparse=WHEN            control creation of sparse files
. See below
      --strip-trailing-slashes  remove any trailing slashes fro
m each SOURCE
                                 argument
  -s, --symbolic-link          make symbolic links instead of c
opying
  -S, --suffix=SUFFIX          override the usual backup suffix
  -t, --target-directory=DIRECTORY  copy all SOURCE arguments i
nto DIRECTORY
  -T, --no-target-directory    treat DEST as a normal file
  -u, --update                 copy only when the SOURCE file i
s newer
                                 than the destination file or w
hen the
                                 destination file is missing
  -v, --verbose                explain what is being done
  -x, --one-file-system        stay on this file system
  -Z                           set SELinux security context of
destination
                                 file to default type
      --context[=CTX]          like -Z, or if CTX is specified
then set the
                                 SELinux or SMACK security cont
ext to CTX
      --help     display this help and exit
      --version  output version information and exit

By default, sparse SOURCE files are detected by a crude heurist
ic and the
corresponding DEST file is made sparse as well.  That is the be
havior
selected by --sparse=auto.  Specify --sparse=always to create a
 sparse DEST
file whenever the SOURCE file contains a long enough sequence o
f zero bytes.
Use --sparse=never to inhibit creation of sparse files.

When --reflink[=always] is specified, perform a lightweight cop
y, where the
data blocks are copied only when modified.  If this is not poss
ible the copy
fails, or if --reflink=auto is specified, fall back to a standa
rd copy.
Use --reflink=never to ensure a standard copy is performed.

The backup suffix is '~', unless set with --suffix or SIMPLE_BA
CKUP_SUFFIX.
The version control method may be selected via the --backup opt
```

```
The version control method may be selected via the --backup opt
ion or through
the VERSION_CONTROL environment variable.  Here are the values:

  none, off       never make backups (even if --backup is given
)
  numbered, t     make numbered backups
  existing, nil   numbered if numbered backups exist, simple ot
herwise
  simple, never   always make simple backups

As a special case, cp makes a backup of SOURCE when the force a
nd backup
options are given and SOURCE and DEST are the same name for an
existing,
regular file.

GNU coreutils online help: <https://www.gnu.org/software/coreut
ils/>
Full documentation <https://www.gnu.org/software/coreutils/cp>
or available locally via: info '(coreutils) cp invocation'
lgj@hzcu-hpct:~/lab3$
```

❌目的地丢了“.”

```
lgj@hzcu-hpct:~/lab3$ cp /public1/soft/IPCC/2022/first/pivot.ta
r .
cp: cannot stat '/public1/soft/IPCC/2022/first/pivot.tar': No s
uch file or directory
lgj@hzcu-hpct:~/lab3$
```

？服务器没有这个文件

```
lgj@hzcu-hpct:~/lab3$ ls /public1/soft/IPCC 2>/dev/null || echo
 "IPCC 目录不存在"
find /public1 -name "pivot.tar" 2>/dev/null | head
IPCC 目录不存在
lgj@hzcu-hpct:~/lab3$ git clone https://github.com/HZCU-HPC-Tea
m/2022_IPCC.git repo
cp repo/2022_IPCC/pivot.tar .
ls -l
Cloning into 'repo'...
```

```
lgj@hzcu-hpct:~/lab3$ git clone https://github.com/HZCU-HPC-Team/2022_IPCC.git repo
cp repo/2022_IPCC/pivot.tar .
ls -l
Cloning into 'repo'...
fatal: unable to access 'https://github.com/HZCU-HPC-Team/2022_IPCC.git/': Failed to connect to github.com port 443 after 130978 ms: Connection timed out
cp: cannot stat 'repo/2022_IPCC/pivot.tar': No such file or directory
total 0
lgj@hzcu-hpct:~/lab3$
```

？服务器连不上GitHub（原因：机房网络出境受限。你的 Mac 校园网能通 GitHub SSH，但服务器不行，这是两台机器各自的网络环境不同。）

>>>**Mac 下载 → scp 传到服务器(scp:在 Mac 和服务器之间**复制，文件真的从一台电脑流到另一台;cp ： 同一台机器内复制)

下载了pivot文件夹之后拖拽到lab3文件夹并上传



终端验证：

ls –l —— 列出 ~/lab3 里面有什么，–l 是"长格式”，除了文件名还显示权限、大小、日期。它回答的问题是：**lab3 里到底是一个还没解的 pivot.tar，还是已经解出来的 pivot 文件夹？** 这决定下一步是先 tar –xf 还是直接编译。

ls pivot —— 列出 pivot 文件夹里面的内容。它回答的问题是：**三个关键文件齐不齐?**

预期要看到：

文件	作用	缺了会怎样
pivot.c	源码	没法编译

uniformvector-2dim-5h.txt	输入数据	程序跑不起来（它从当前目录读这个文件）
refer-2dim-5h.txt	参考答案	最后没法对拍验证

< 4 运行

```
● lgj@hzcu-hpct:~/lab3$ cd pivot
● lgj@hzcu-hpct:~/lab3/pivot$ ./pivot
  dim = 2, n = 500, k = 2
  0 499   0 351 110336.788710     0 23 70432.598952
  1 499   0 351 110336.788710     1 58 60839.591956
  2 499   2 320 111591.674412     1 58 60839.591956
  3 499   3 143 115596.768362     1 58 60839.591956
  4 499   3 143 115596.768362     1 58 60839.591956
```

*二维数据、500 个点、每组合选 2 个支撑点

```
  497 499         143 351 117761.647418   83 226 43769.849602
  498 499         143 351 117761.647418   83 226 43769.849602
  499 499         143 351 117761.647418   83 226 43769.849602
  Using time : 495279.367000 ms
  max : 143 351 117761.647418
  min : 83 226 43769.849602
```

***Using time : 495279.367 ms ≈ 495 秒 ≈ 8.25 分钟——基线成绩**

*max : 143 351 / min : 83 226：算出来的最优/最差支撑点组合

```
● lgj@hzcu-hpct:~/lab3/pivot$ diff result.txt refer-2dim-5h.txt
  && echo "结果一致"
  结果一致
```

第一次的对照物是 refer 文件

```
refer-2dim-5h.txt（出题人提供的"标准答案"）
        ↑ diff 对拍
基线程序（未优化原版）—— 第一层验证：你的环境、编译、运行全流程没毛病
        ↑ 对照
优化程序 —— 第二层验证：你改的代码没有改变计算结果
```

支撑点：英文就是题目名里的 **pivot**。它是一种“参照物”。

泛构：“统一抽象”的框架名，坐标：框架里缺的那块拼图，支撑点：补拼图的方法

*优化

```
⊗ lgj@hzcu-hpct:~/lab3/pivot$ gcc pivot.c -02
  -lm -o pivot
  gcc: error: unrecognized command-line option
   '-02'
```

o打成0

```
lgj@hzcu-hpct:~/lab3/pivot$ gcc pivot.c -O2
-lm -o pivot
pivot.c: In function 'main':
pivot.c:162:5: warning: ignoring return valu
e of 'fscanf' declared with attribute 'warn_
unused_result' [-Wunused-result]
  162 |     fscanf(file, "%d", &dim);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~
pivot.c:163:5: warning: ignoring return valu
e of 'fscanf' declared with attribute 'warn_
unused_result' [-Wunused-result]
  163 |     fscanf(file, "%d", &n);
      |     ^~~~~~~~~~~~~~~~~~~~~~
pivot.c:164:5: warning: ignoring return valu
e of 'fscanf' declared with attribute 'warn_
unused_result' [-Wunused-result]
  164 |     fscanf(file, "%d", &k);
      |     ^~~~~~~~~~~~~~~~~~~~~~
pivot.c:177:13: warning: ignoring return val
ue of 'fscanf' declared with attribute 'warn
_unused_result' [-Wunused-result]
  177 |             fscanf(file, "%lf", &coo
rd[i*dim + j]);
      |             ^~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~
```

```
lgj@hzcu-hpct:~/lab3/pivot$ ./pivot
dim = 2, n = 500, k = 2
```

```
499 499         143 351 117761.647418   83 2
26 43769.849602
Using time : 71800.056000 ms
max : 143 351 117761.647418
min : 83 226 43769.849602
```

-o2（gcc 的二级优化开关，Optimization：优化） 71.8秒（6.9倍）

```
lgj@hzcu-hpct:~/lab3/pivot$ diff result.txt
refer-2dim-5h.txt && echo "结果一致"
结果一致
```

2.说清“最小单元”

#从源码到结果的最小单位

##1.环境

做了：which gcc、gcc --version、dpkg -l | grep build-essential

确认：gcc 12.3.0 存在，编译套装齐全

##2.源码

做了：获得 pivot.tar(clone 超时失败， 浏览器下载 + 拖拽到服务器)，tar -xf解包，ls 验货三个文件

确认：pivot.c / 输入数据 / refer 三个文件都在

##3.编译

做了：gcc pivot.c -lm -o pivot

确认：gcc 沉默 = 无语法错误；ls 看见 pivot 可执行文件

##4.链接

做了：gcc 默认连坐，-lm 负责到 libm 找pow/sqrt 的实现

确认：没有 underfined reference 报错 = 链接成功

##5.运行

做了：./pivot （必须 cd 进 pivot 目录，程序从当前目录读数据）

确认：打印 dim = 2, n = 500,k = 2 开始计算，进程结束回到提示符

##6.输出

做了：程序打两个 Using time，写 result.txt

确认：diff result.txt refer-2dim-5h.txt 无差异 = 输出正确

步骤	输入	动作	产物	怎么确认
环境	一台空机器	查有没有编译器	确认 gcc 12.3.0	which / --version
源码	一个 tar 包	解包	三个文件	ls 看齐不齐
编译	pivot.c	gcc 翻译	pivot.o	无 error
链接	pivot.o + libm	拼装	pivot 可执行文件	无 undefined reference
运行	pivot + 数据	交给 CPU	计算结果	打印 dim/n/k 开始算

输出	计算结果	写文件	result.txt	diff 与 refer 一致

① **编译（compile）** ——gcc 干的第一件事。把 pivot.c（人能看懂的 C 代码）翻译成机器码，产出 .o 目标文件。但此时它只是个“碎片”：你的代码里调了 pow、sqrt、malloc，这些函数的机器码不在你的文件里，只有“调用申请”。这一步检查的是**语法**——你昨天担心的“埋雷”就是在这一步被拦下的。

② **链接（link）** ——把你的碎片和**库**里现成的机器码拼装成一个完整可执行文件。-lm 的意思就是“去 libm（math 库）里把 pow、sqrt 的实现找出来装上”。这一步检查的是**函数存不存在**——如果链接时找不到某个函数，报 undefined reference。

③ **运行（run）** ——./pivot 敲下去。要点：**指令本身不计算，它只是让操作系统把 pivot 这个文件加载进内存、交给 CPU 开始执行**。真正干活的从这一刻起是那个进程，不是你的终端——所以它跑 8 分钟时你的终端“没反应”但没死，是程序在算。算完它自己写 result.txt、打印时间，进程退出，控制权回到 shell。

3. 编译优化实验（不改代码）

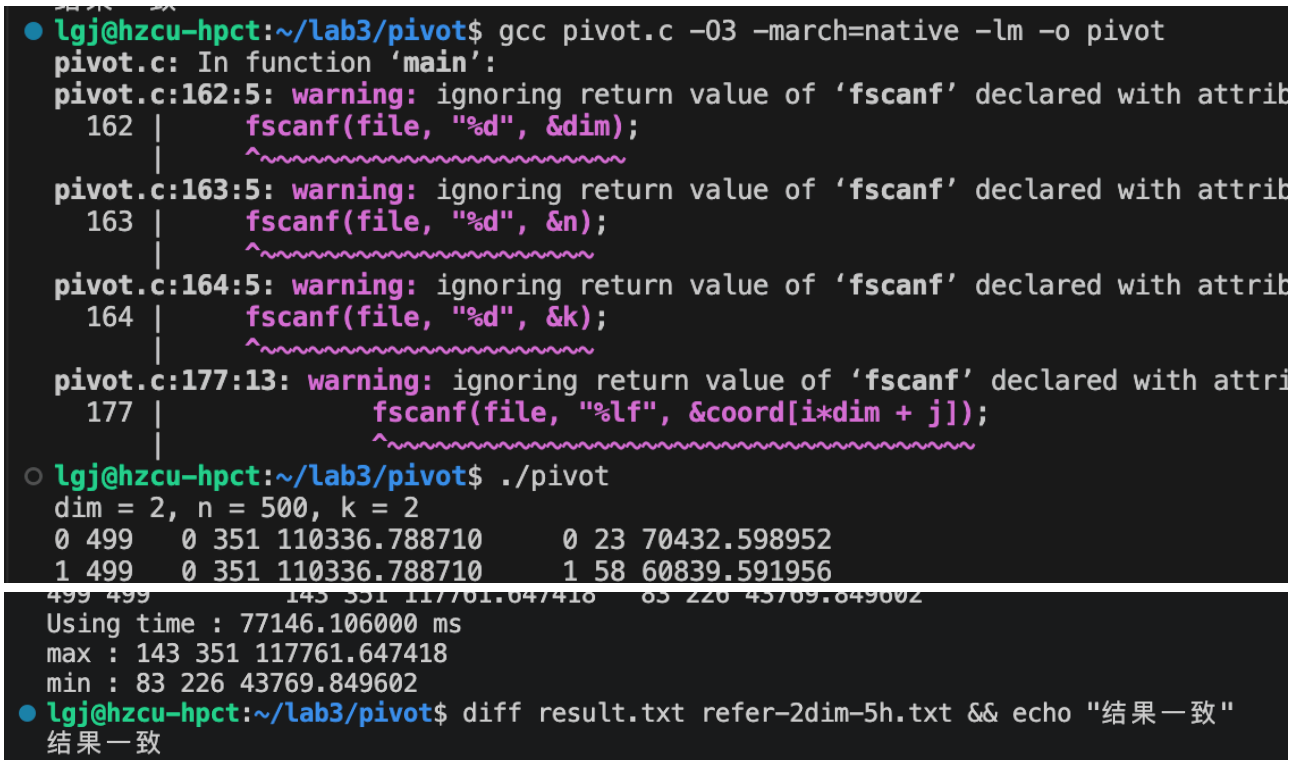
- 只换编译选项：-O0 / -O1 / -O2 / -O3（可加 -march=native），每种跑 1—3 次；
- 记录时间、算加速比；

换选项后再 diff 一次，确认结果没变。

-o1

```
lgj@hzcu-hpct:~/lab3/pivot$ gcc pivot.c -O1 -lm -o pivot
pivot.c: In function ‘main’:
pivot.c:162:5: warning: ignoring return value of ‘fscanf’ dec
  162 |     fscanf(file, "%d", &dim);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~
pivot.c:163:5: warning: ignoring return value of ‘fscanf’ dec
  163 |     fscanf(file, "%d", &n);
      |     ^~~~~~~~~~~~~~~~~~~~~~
pivot.c:164:5: warning: ignoring return value of ‘fscanf’ dec
  164 |     fscanf(file, "%d", &k);
      |     ^~~~~~~~~~~~~~~~~~~~~~
pivot.c:177:13: warning: ignoring return value of ‘fscanf’ de
  177 |             fscanf(file, "%lf", &coord[i*dim + j]);
      |             ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
lgj@hzcu-hpct:~/lab3/pivot$ ./pivot
dim = 2, n = 500, k = 2
0 499   0 351 110336.788710     0 23 70432.598952
1 499   0 351 110336.788710     1 58 60839.591956
499 499     143 351 117761.647418   83 226 43769.849602
Using time : 98480.124000 ms
max : 143 351 117761.647418
min : 83 226 43769.849602
lgj@hzcu-hpct:~/lab3/pivot$ diff result.txt refer-2dim-5h.txt && echo "结果一致"
结果一致
lgj@hzcu-hpct:~/lab3/pivot$
```

-o3 -march=native



编译选项	Using time (ms)	加速比(÷495279)	结果一致
-O0	495279	1.00×	✓
-O1	98480	5.03×	✓
-O2	71800	6.90×	✓
-O3	56389	8.78×	✓
-O3 -march=native	63429 / 67913 / 77146，**取最小 63429**	7.81×	✓

4. perf

perf 是什么、stat / record / report 各干什么

perf : Linux 自带的**性能分析器**，相当于给程序做体检的仪器。程序跑得慢，慢在哪？perf 让 CPU 的硬件计数器（统计时钟周期数、指令数、缓存命中、分支预测等）告诉你答案。它不改你的代码，只是在旁边“观察”。

命令：

perf stat:

体检报告：整体计数（cycles、instructions、cache-misses、branch-misses）;判断程序是"计算密集”还是“访存密集"

perf record:

录像：采样记录 CPU 时间花在哪些函数上;生成 perf.data

perf report:

回放录像：按耗时占比列出函数排行;找出热点函数

🔔**stat 看“全身指标”，record + report 看“病灶在哪”。**

🔔**#整体计数**

perf 数出来的“总账数字”，不是某个函数、某一行代码的，而是**整个程序跑这一趟的总和**：

- cycles：CPU 消耗了多少个时钟周期（1 个周期 ≈ 0.3 纳秒，这个数就是“总耗时”的另一种说法）
- instructions：一共执行了多少条机器指令
- cache-misses：去内存取数据有多少次没在缓存里找到（“扑空”次数）
- branch-misses：if/for 的条件判断，CPU 猜错分支多少次（猜错要清空流水线，代价大）

它跟后面 record 的区别：**stat 只给你总数，不告诉你是哪段代码贡献的**。像医院体检报告——血压高、血糖高都告诉你了，但不说为啥高。

#计算密集 vs 访存密集

程序慢，无非两种原因：

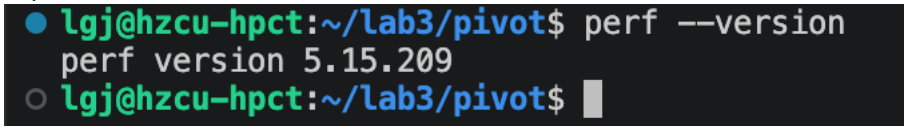
- 计算密集（compute-bound）：CPU 一直在算，算不过来。瓶颈是“算术/逻辑指令太多”。
- 访存密集（memory-bound）：CPU 大部分时间在等数据——数据在内存/硬盘里，取一次要等几百个周期，CPU 干等着。

为什么区分？因为解法完全不同：计算密集要想办法少算（换算法、向量化、并行）；访存密集要想办法少取/快取（改数据结构、提高缓存命中、分块）。搞错方向，优化白干。

#perf.data

perf record 生成的录像文件**，二进制格式，路径默认是当前目录的 perf.data。里面存的是采样记录：每隔一小段时间（默认约 1ms）打断程序一次，记下“此刻 CPU 在跑哪个函数、哪行指令”，跑 56 秒就能攒下上万条样本。

perf report 读的就是这个文件，把样本按函数汇总成百分比排行——样本多的函数 = 耗时占比大 = 热点**。perf.data 一般几 MB，不用提交，也不用看（二进制看不了），它只是 record 和 report 之间的中转文件。



判断服务器是否有依赖：有 perf version 5.15.209

```
lgj@hzcu-hpct:~/lab3/pivot$ gcc pivot.c -O3 -lm -o pivot
pivot.c: In function 'main':
pivot.c:162:5: warning: ignoring return value of 'fscanf' declared with attribute 'warn_unused_result' [-Wunused-result]
  162 |     fscanf(file, "%d", &dim);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~
pivot.c:163:5: warning: ignoring return value of 'fscanf' declared with attribute 'warn_unused_result' [-Wunused-result]
  163 |     fscanf(file, "%d", &n);
      |     ^~~~~~~~~~~~~~~~~~~~~~
pivot.c:164:5: warning: ignoring return value of 'fscanf' declared with attribute 'warn_unused_result' [-Wunused-result]
  164 |     fscanf(file, "%d", &k);
      |     ^~~~~~~~~~~~~~~~~~~~~~
pivot.c:177:13: warning: ignoring return value of 'fscanf' declared with attribute 'warn_unused_result' [-Wunused-result]
  177 |             fscanf(file, "%lf", &coord[i*dim + j]);
      |             ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
lgj@hzcu-hpct:~/lab3/pivot$ perf stat ./pivot > sta_out.txt 2> perf_stat.log

cat perf_stat.log

lgj@hzcu-hpct:~/lab3/pivot$
lgj@hzcu-hpct:~/lab3/pivot$ cat perf_stat.log

 Performance counter stats for './pivot':

         56,225.43 msec task-clock                #    1.000 CPUs utilized
               233      context-switches          #    4.144 /sec
                11      cpu-migrations            #    0.196 /sec
               116      page-faults               #    2.063 /sec
   146,176,900,552      cycles                    #    2.600 GHz
   975,027,320,053      instructions              #    6.67  insn per cycle
   157,334,727,312      branches                  #    2.798 G/sec
        65,119,142      branch-misses             #    0.04% of all branches
   871,882,061,190      slots                     #   15.507 G/sec
   837,690,607,810      topdown-retiring          #     96.5% retiring
     6,838,290,676      topdown-bad-spec          #      0.8% bad speculation
    17,095,726,690      topdown-fe-bound          #      2.0% frontend bound
     6,880,922,425      topdown-be-bound          #      0.8% backend bound
                 0      topdown-heavy-ops         #      0.0% heavy operations       #     96.5% light operations
     6,838,290,676      topdown-br-mispredict     #      0.8% branch mispredict      #      0.0% machine clears
     6,838,290,676      topdown-fetch-lat         #      0.8% fetch latency          #      1.2% fetch bandwidth
     3,483,537,389      topdown-mem-bound         #      0.4% memory bound           #      0.4% Core bound

      56.233121493 seconds time elapsed

      56.217995000 seconds user
       0.007999000 seconds sys
```

指标	数值	含义	判定
task-clock	56225 ms，**1.000 CPUs utilized**	恰好用满 1 个核，没有等待 IO	纯 CPU 计算 ✓
cycles	1461.8 亿 @ 2.6 GHz	总周期数，÷2.6GHz ≈ 56.2 秒，和耗时对上了	✓
instructions	9750 亿条	一共干了这么多活	✓
IPC	6.67 insn/cycle	每周期干 6.67 条活——**极高**	计算密集实锤
branch-misses	**0.04%**	分支预测几乎全对	不是瓶颈
topdown-mem-bound	**0.4%**	整个时间里只有 0.4% 在等内存	**访存几乎为零**
topdown-retiring	**96.5%**	96.5% 的周期都在“出活”（干有效计算）	CPU 效率极高

IPC = instructions ÷ cycles = 9750 亿 ÷ 1462 亿 ≈ **6.67**

🔔**IPC = Instructions Per Cycle，每个时钟周期执行完的指令条数。**衡量 **CPU 利用效率的指标。**
IPC 高 = CPU 没在停等 = 计算密集的旁证；IPC 低（比如 0.3）+ cache-miss 高 = CPU 老在等数据 = 访存密集。

IPC	含义
≈ 0.2~0.5	CPU 大部分周期在停工（等内存、依赖链卡死、缓存全 miss）
≈ 1	一般水平，每周期完成约 1 条
2~4	流水线喂得饱，优化良好
≥ 4	极高——说明 CPU 同时在执行多条指令，通常有向量化参与

```
lgj@hzcu-hpct:~/lab3/pivot$ perf stat -e cache-references,cache-misses ./pivot
458 499         143 351 117761.647418   83 226 43769.849602
459 499         143 351 117761.647418   83 226 43769.849602
460 499         143 351 117761.647418   83 226 43769.849602
461 499         143 351 117761.647418   83 226 43769.849602
462 499         143 351 117761.647418   83 226 43769.849602
463 499         143 351 117761.647418   83 226 43769.849602
464 499         143 351 117761.647418   83 226 43769.849602
465 499         143 351 117761.647418   83 226 43769.849602
466 499         143 351 117761.647418   83 226 43769.849602
467 499         143 351 117761.647418   83 226 43769.849602
468 499         143 351 117761.647418   83 226 43769.849602
469 499         143 351 117761.647418   83 226 43769.849602
470 499         143 351 117761.647418   83 226 43769.849602
471 499         143 351 117761.647418   83 226 43769.849602
472 499         143 351 117761.647418   83 226 43769.849602
473 499         143 351 117761.647418   83 226 43769.849602
474 499         143 351 117761.647418   83 226 43769.849602
475 499         143 351 117761.647418   83 226 43769.849602
476 499         143 351 117761.647418   83 226 43769.849602
477 499         143 351 117761.647418   83 226 43769.849602
478 499         143 351 117761.647418   83 226 43769.849602
479 499         143 351 117761.647418   83 226 43769.849602
480 499         143 351 117761.647418   83 226 43769.849602
481 499         143 351 117761.647418   83 226 43769.849602
482 499         143 351 117761.647418   83 226 43769.849602
483 499         143 351 117761.647418   83 226 43769.849602
484 499         143 351 117761.647418   83 226 43769.849602
485 499         143 351 117761.647418   83 226 43769.849602
486 499         143 351 117761.647418   83 226 43769.849602
487 499         143 351 117761.647418   83 226 43769.849602
488 499         143 351 117761.647418   83 226 43769.849602
489 499         143 351 117761.647418   83 226 43769.849602
490 499         143 351 117761.647418   83 226 43769.849602
491 499         143 351 117761.647418   83 226 43769.849602
492 499         143 351 117761.647418   83 226 43769.849602
493 499         143 351 117761.647418   83 226 43769.849602
494 499         143 351 117761.647418   83 226 43769.849602
495 499         143 351 117761.647418   83 226 43769.849602
496 499         143 351 117761.647418   83 226 43769.849602
497 499         143 351 117761.647418   83 226 43769.849602
498 499         143 351 117761.647418   83 226 43769.849602
499 499         143 351 117761.647418   83 226 43769.849602
Using time : 55899.886000 ms
max : 143 351 117761.647418
min : 83 226 43769.849602

 Performance counter stats for './pivot':

           937,345      cache-references
           203,995      cache-misses              #   21.763 % of all cache refs

      55.901999386 seconds time elapsed

      55.886944000 seconds user
       0.003999000 seconds sys
```

cache-miss 率 21.76%，但 cache-references 总量仅 93.7 万（vs 1462 亿总周期），miss 实际耗时占比 <0.1%；

topdown memory-bound 0.4% 相互印证。

结论：访存绝对开销可忽略，计算密集型判定不变。

🔔性能分析的重要素养：**百分比必须带着绝对量看**

绝对量："数出来的真实数量"，和它对立的是相对量（百分比） 。

绝对量：cache-misses = **203,995 次**（数出来的真实次数）；总周期 = **1,462 亿**（真实数量）

相对量：miss 率 = 21.76%（两个绝对量相除得来的比例）

【百分比回答“占多大比例”，绝对量回答“实际造成多大损失”】

「日常类比：打折 50%（相对量）听着大，商品原价 2 块（绝对量），省 1 块，路上车费 3 块——不划算。看相对量心动，算绝对量才做对决定。」

命令-参数

- perf stat ./pivot（默认参数，各计数器 1 次）
- perf stat -e cache-references,cache-misses ./pivot（显式指定缓存事件，1 次）
- perf record ./pivot（默认采样，约 1ms 一次，得 226,139 个样本，0 丢失）
- perf report --stdio | head -30

环境：Ubuntu + GCC 12.3.0，-O3 编译，单节点单线程